

TD/TP 2 – Activité 2

Style Pipe & Filtre
Architectures Logicielles – SIQ1

HIMEUR Idris– DIAR Adem

2025/2026

Table des matières

1	Introduction	2
2	Description de l’architecture réalisée	2
2.1	Principe général	2
2.2	Les composants	2
2.3	Les pipes et les messages	2
2.4	Flux d’exécution	3
	Note importante – Composants frontières et couplage GUI	3
3	Style d’échange utilisé	3
4	Vues de l’architecture	4
4.1	Vue de développement (vue des modules)	4
4.2	Vue C&C (Composants et Connecteurs)	4
5	Comparaison Activité 1 vs Activité 2	5

1 Introduction

L'objectif de cette activité est de transformer l'application calculatrice développée lors de l'Activité 1 en une architecture à base du style **Pipe & Filtre**. Le code de calcul, de persistance et d'interface graphique est conservé à l'identique ; seule la *structure de communication* entre les composants change.

2 Description de l'architecture réalisée

2.1 Principe général

Dans le style Pipe & Filtre, les composants (*filtres*) sont des unités de traitement indépendantes qui ne communiquent qu'à travers des canaux de données (*pipes*). Chaque filtre :

- lit ses données d'entrée depuis son **inPipe**,
- effectue son traitement,
- publie son résultat dans son **outPipe**.

Aucun filtre ne connaît l'identité de ses voisins ; il ne voit que ses deux pipes.

2.2 Les composants

Composant	Responsabilités
GuiSourceFilter	Lit les saisies de l'utilisateur (opérandes, opération), construit un CalculationRequest et le pousse dans requestPipe .
ComputeFilter	Calcul de la somme, du produit, du factoriel, génération d'un nombre aléatoire.
TraceFilter	Sauvegarde de l'opération dans un fichier texte, chargement et restitution de la trace persistée.
GuiOutputAdapter	Adaptateur frontière : tire le TraceRecord final du tracePipe et met à jour l'interface graphique.

2.3 Les pipes et les messages

Trois pipes relient les composants. Chaque pipe est typé selon le message qu'il transporte :

Pipe	Type de message	Producteur	Consommateur
requestPipe	CalculationRequest	GuiSourceFilter	ComputeFilter
resultPipe	CalculationResult	ComputeFilter	TraceFilter
tracePipe	TraceRecord	TraceFilter	GuiOutputAdapter

2.4 Flux d'exécution

Lorsque l'utilisateur clique sur un bouton, **GuiSourceFilter** est déclenché : il lit les champs de saisie, pousse un **CalculationRequest** dans **requestPipe**, puis appelle **pipeline.run()** qui va lancé par la suite **ComputeFilter**, **TraceFilter** et **GuiOutputAdapter**.

ComputeFilter tire la requête, calcule le résultat et pousse un **CalculationResult** dans **resultPipe**. **TraceFilter** tire ce résultat, le persiste et pousse un **TraceRecord** dans **tracePipe**. Enfin, **GuiOutputAdapter** tire le **TraceRecord** et met à jour l'affichage.

Note importante – Composants frontières et couplage GUI

Le style Pipe & Filtre est conçu pour le traitement de flux de données : chaque filtre est une unité de transformation indépendante, sans connaissance du monde extérieur. Les données circulent dans un seul sens : elles entrent par un filtre source et sortent par un filtre sink.

Dans une application à interface graphique, certaines opérations (la lecture des champs de saisie et mise à jour de l'affichage) ne correspondent pas à des transformations de données mais à des interactions avec un système externe. Dans ce type d'architecture, ces interactions sont généralement placées dans des **composants frontières**.

Dans notre pipeline, ces responsabilités sont réparties entre deux filtres frontières :

- **GuiSourceFilter** : filtre d'entrée qui lit les données saisies par l'utilisateur dans **MainGUI** et les injecte dans le pipeline via **requestPipe**.
- **GuiOutputAdapter** : filtre de sortie qui récupère le résultat final depuis **tracePipe** et met à jour l'interface graphique.

Ces filtres servent de frontière entre l'interface graphique et le pipeline de traitement, tandis que les filtres internes (**ComputeFilter** et **TraceFilter**) restent totalement indépendants de la GUI.

La chaîne de traitement est **strictement acyclique** : **GuiSourceFilter** ne reçoit rien du pipeline et **GuiOutputAdapter** n'y envoie rien. Le flux de données suit un sens unique :

```
GuiSourceFilter requestPipe ComputeFilter resultPipe TraceFilter tracePipe
                               GuiOutputAdapter
```

L'orchestration est confiée à **CalculatorPipeline**, responsable du câblage des pipes et de l'initialisation du flux de traitement. L'exécution est déclenchée par l'injection d'un message dans le pipeline via **GuiSourceFilter**. Chaque filtre lit les données depuis son **inPipe** et publie son résultat dans son **outPipe**. Aucun filtre ne connaît ses voisins ; seul le coordinateur définit la structure globale du pipeline.

3 Style d'échange utilisé

Le style d'échange utilisé est le **PIPE** : les données circulent sous forme de **flux unidirectionnel** entre les composants. Chaque producteur **pousse** (push) son message dans le pipe ; le consommateur suivant le **tire** (pull) pour le traiter. Les filtres ne se connaissent pas directement – seuls les pipes les relient.

4 Vues de l'architecture

4.1 Vue de développement (vue des modules)

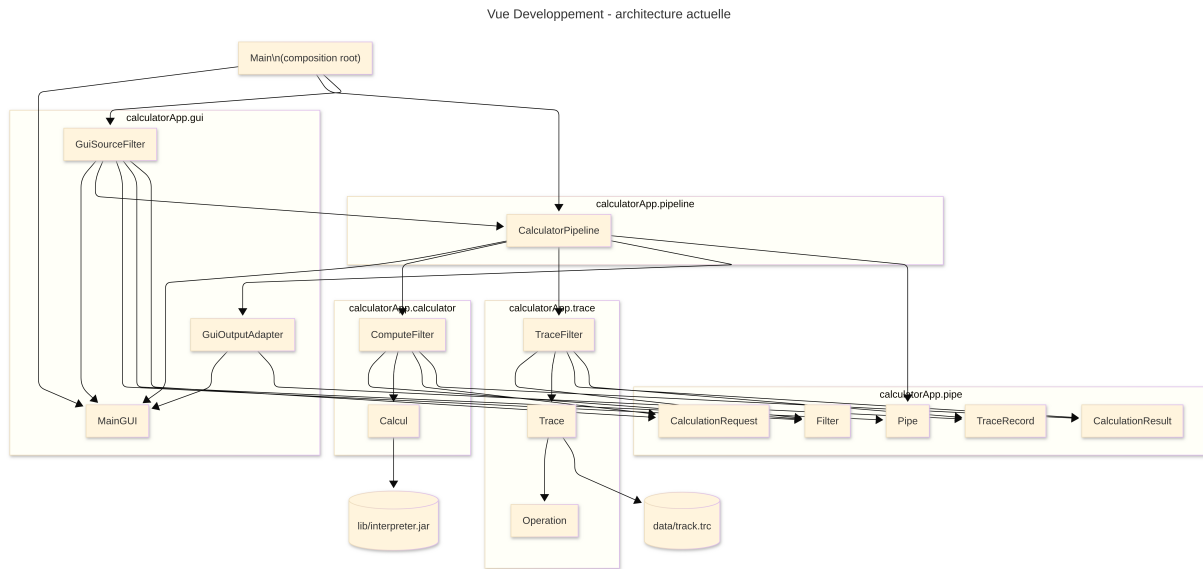


FIGURE 1 – Vue de développement

4.2 Vue C&C (Composants et Connecteurs)

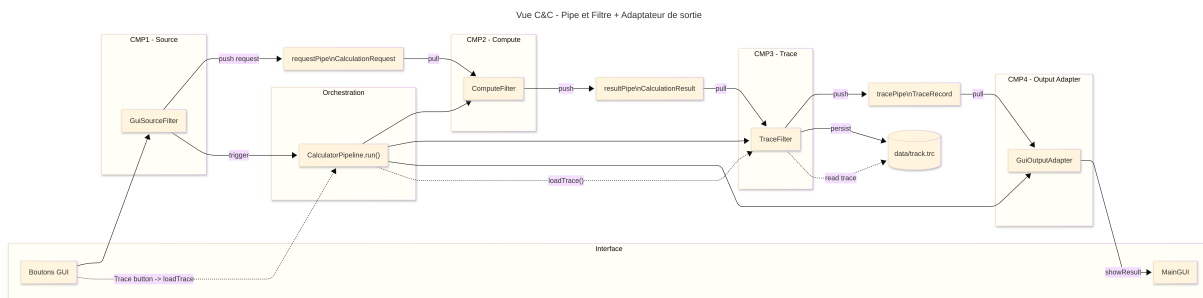


FIGURE 2 – Vue C&C – Pipe & Filtre

5 Comparaison Activité 1 vs Activité 2

Activité 1 – Appel de méthode direct	Activité 2 – Pipe & Filtre
(+) Performance : pas de sérialisation ni de tampon intermédiaire; l'appel est synchrone et instantané.	(+) Indépendance des composants : aucun filtre ne connaît ses voisins; on peut remplacer CMP2 sans toucher CMP1 ni CMP3.
(+) Simplicité de la structure : une seule classe centrale relie tout.	(+) Lisibilité architecturale : la vue C&C montre immédiatement le flux de données et les responsabilités.
(-) Couplage fort : la GUI connaît directement <code>Calcul</code> et <code>Trace</code> ; tout changement se propage.	(+) Réutilisabilité et testabilité : chaque filtre peut être testé isolément avec un pipe bouchon.
(-) Difficile à faire évoluer : ajouter une étape de traitement implique de modifier la GUI ou le coordinateur.	(+) Extensibilité : ajouter un filtre revient à créer une nouvelle classe et à l'insérer dans la chaîne de pipes.
	(-) Transformations interactives difficiles : le retour du résultat vers la GUI nécessite un pipe supplémentaire (<code>tracePipe</code> → <code>GuiFilter</code>), ce qui complexifie légèrement le câblage.
	(-) Overhead léger : création d'objets messages à chaque opération (minime en pratique).

Conclusion. L'Activité 1 est plus performante et plus simple pour une petite application. L'Activité 2 est architecturalement plus robuste : les composants sont vraiment indépendants, le flux de données est explicite, et l'application peut évoluer sans modifier les composants existants.